Praktikum 1 - Matakuliah Pilihan Web (API Development)
Program Studi: Teknik Informatika

Lakukan praktikum dibawah ini, dan buat screenshot untuk pembuktian mengerjakan setiap poin dengan mengisi tabel dibawah, kemudian tunjukan hasil akhir dari men-share repository github yang telah dibuat.

A. Setup Laravel dan Mysql

1. Buat project laravel baru dengan composer dengan nama laravel_api (Gunakan versi 12.x atau yang lebih baru)
2. Mempersiapkan database dan buat database baru bernama dblaravelapi
3. Update .env dan hubungkan dengan nama database dblaravelapi
4. Lakukan migrasi dengan menggunakan php artisan migrate
5. Pastikan laravel sudah terhubung dengan tabel dengan mengecek tabel yang dibuat laravel dalam mysql

B. Membuat Controller

1. Buatlah sebuah controller dengan nama Api/ProductController
   php artisan make:controller Api/ProductController
   (Pastikan ada di dalam folder Api)
   Pada bootstrap/app.php tambahkan setelah code web: __DIR
   api: __DIR__.'/../routes/api.php',

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
```

2. Tambahkan Routes baru di router/api.php
   Route::get('/products', [ProductController::class, 'index']);
   (Apabila file api.php belum ada, clone web.php dan hapus routes "/")
3. Test API dengan mengunjungi localhost:8000/api/products

C. Gunakan database dan Model

1. Pastikan database sudah terkoneksi dengan laravel.
2. Buatkan sebuah tabel dengan nama tblproducts dengan field (id, nama, deskripsi, foto, dan harga) untuk tipe data menyesuaikan.

3. Tambahkan minimal 10 data untuk produk dengan nama2 product elektronik seperti tv, kulkas, ac, dan lain sebagainya.
4. Buat model bernama Product
   php artisan make:model Product
5. Hubungkan model Product dengan tblproduct dengan mengisi pada class Product

```php
class Product extends Model
{
    protected $table = 'tblproducts';
    protected $primaryKey = 'id';
    public $timestamps = false;

    // Field yang boleh diisi
    protected $fillable = [
        'nama',
        'deksripsi',
        'foto',
        'harga',
    ];

}
```

6. Pada Product controller ubah dalam fungsi index, dapatkan data dari model Product
   Dengan menambahkan $products = Product:all();
   Dan lakukan return data => $products

D. Membuat CRUD standar API Resource

1. Ubah api.php dengan api Resource yang sesuai standar laravel

```php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\ProductController;

// Route::get('/products', [ProductController::class, 'index']);

Route::apiResource('products', ProductController::class);
```

Dengan apiResource kamu tidak perlu menambahkan routes satu persatu untuk ProductController seperti create, update, delete, search dan view. Namun ada syarat penamaan fungsi agar routes ini bekerja yaitu **index, show, store, update, dan destroy.**

2. Ubah ProductController untuk fungsi-fungsi berikut **index, show, store, update, dan destroy.**

```php
// 🔹 GET /products
public function index()
{
    $products = Product::all();

    return response()->json([
        'status' => true,
        'data' => $products
    ]);
}
```

Gambar 04: Fungsi Index pada ProductsController

```php
// 🔹 GET /products/{id}
public function show($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    return response()->json([
        'status' => true,
        'data' => $product
    ]);
}
```

Gambar 05: Fungsi Show pada ProductsController

```php
// 🔹 POST /products
public function store(Request $request)
{
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);

    $product = Product::create($request->all());

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil ditambahkan',
        'data' => $product
    ]);
}
```

Gambar 06: Fungsi Store pada ProductsController

```php
// 🔹 PUT /products/{id}
public function update(Request $request, $id)
{
    $product = Product::find($id);
    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);
    $product->update($request->all());
    return response()->json([
        'status' => true,
        'message' => 'Product berhasil diupdate',
        'data' => $product
    ]);
}
```

Gambar 07: Fungsi Update pada ProductsController

```php
// 🔹 DELETE /products/{id}
public function destroy($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    $product->delete();

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil dihapus'
    ]);
}
```

Gambar 08: Fungsi Destroy pada ProductsController

E. API Documentation

1. Install Scramble dengan menggunakan composer
   `composer require dedoc/scramble`

2. Masukan ke vendor laravel project anda
   `php artisan vendor:publish --provider="Dedoc\Scramble\ScrambleServiceProvider"`

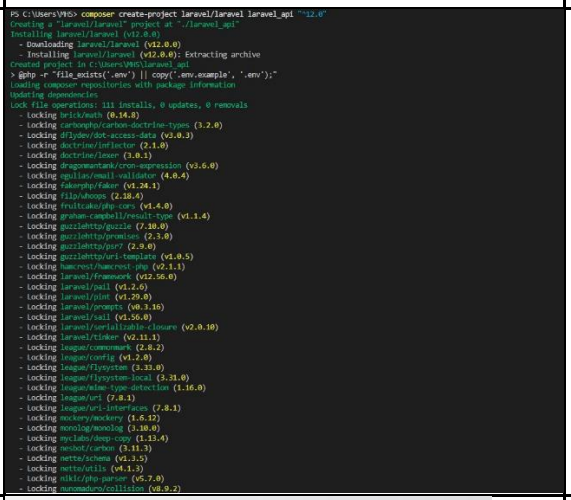
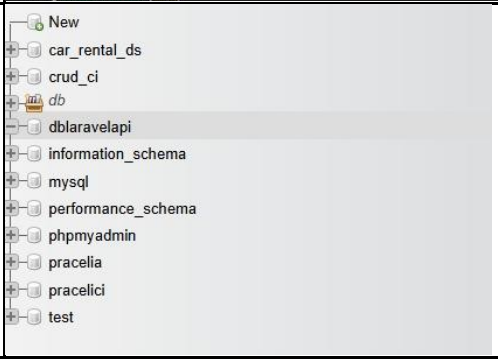
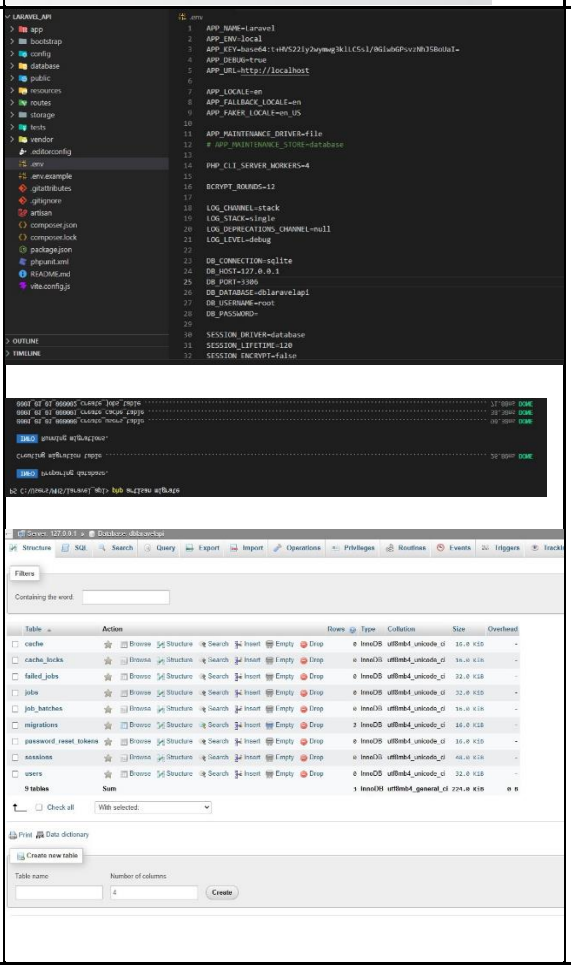
3. Jalankan server (artisan serve)
4. Akses API documentation pada browser ke http://localhost:8000/docs/api
5. Untuk mendambahkan deksripsi setiap API Endpoints bisa menggunakan DocBlock diatas fungsi dalam controller
6. Menambahkan DocBlock (Title dan Description)

```php
class ProductController extends Controlle
{

/**
 * Ambil semua produk.
 *
 * Endpoint ini mengembalikan seluruh data produk yang tersedia.
 */

    // 🔹 GET /products
    public function index()
    {
        $products = Product::all();

        return response()->json([
            'status' => true,
            'data' => $products
        ]);
    }
```

Gambar 08: DocBlock untuk title node index dan deksripsi

7. Refresh browser ke http://localhost:8000/docs/api
8. Tambahkan Title dan Deskripsi pada semua Endpoints

Hasil Pengerjaan

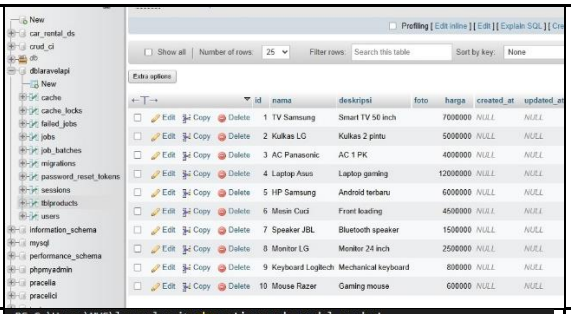
No.	Instruksi	Screenshot	Kendala/Saran
A.	**Setup Laravel dan Mysql**		
1.	Buat project laravel baru dengan composer dengan nama laravel_api (Gunakan versi 12.x atau yang lebih baru)		-
2.	Mempersiapkan database dan buat database baru bernama dblaravelapi		-
3.	3. Update .env dan hubungkan dengan nama database dblaravelapi 4. Lakukan migrasi dengan menggunakan php artisan migrate		-

B.	**Membuat Controller**		

1.	Buatlah sebuah controller dengan nama Api/ProductController php artisan make:controller Api/ProductController (Pastikan ada di dalam folder Api) Pada bootstrap/app.php tambahkan setelah code web: __DIR api: __DIR__.'/../routes/api.php'	PS C:\Users\MHS\laravel_api> php artisan make:controller Api/ProductController
		INFO Controller [C:\Users\MHS\laravel_api\app\Http\Controllers\Api\ProductController.php] created successful
		PS C:\Users\MHS\laravel_api>[]
		bootstrap > app.php
		1 <?php
		2
		3 use Illuminate\Foundation\Application;
		4 use Illuminate\Foundation\Configuration\Exceptions;
		5 use Illuminate\Foundation\Configuration\Middleware;
		6
		7 return Application::configure(basePath: dirname(__DIR__))
		8 ->withRouting(
		9 web: __DIR__.'/../routes/web.php',
		10 api: __DIR__.'/../routes/api.php',
		11 commands: __DIR__.'/../routes/console.php',
		12 health: '/up',
		13)
		14 ->withMiddleware(function (Middleware $middleware) {
		15 //
		16 })
		17 ->withExceptions(function (Exceptions $exceptions) {
		18 //
		19 })->create();
			-

2.	Tambahkan Routes baru di router/api.php Route::get('/products', [ProductController::class, 'index']); (Apabila file api.php belum ada, clone web.php dan hapus routes “/”)	routes api.php console.php web.php routes > api.php 1 <?php 2 3 use Illuminate\Support\Facades\Route; 4 use App\Http\Controllers\Api\ProductController; 5 6 Route::get('/products', [ProductController::class, 'index']);	-
3.	Test API dengan mengunjungi localhost:8000/api/products	localhost:8000/api/products Pretty-print {"message":"List products"}	-
C.	**Gunakan Database dan Model**		

1.	Buatkan sebuah tabel dengan nama tblproducts dengan field (id, nama, deskripsi, foto, dan harga) untuk tipe data menyesuaikan.	database > migrations > 2026_04_06_071939_create_tblproducts_table.php
		8 {
		9 /**
		11 */
		12 public function up(): void
		13 {
		14 Schema::create('tblproducts', function (Blueprint $table) {
		15 $table->id();
		16 $table->string('nama');
		17 $table->text('deskripsi');
		18 $table->string('foto')->nullable();
		19 $table->integer('harga');
		20 $table->timestamps();
		21 });
		22 }
		23
		24 /**
			-

2.	Tambahkan minimal 10 data untuk produk dengan nama2 product elektronik seperti tv, kulkas, ac, dan lain sebagainya.	 Profiling [Edit inline] [Edit] [Explain SQL] [Cre Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None Extra options ← T → New car_rental_ds crud_ci db dblaravelapi New cache cache_locks failed_jobs jobs job_batches migrations password_reset_tokens sessions tblproducts users information_schema mysql performance_schema phpmyadmin pracella pracellcl Edit Copy Delete 1 TV Samsung Smart TV 50 inch 7000000 NULL NULL Edit Copy Delete 2 Kulkas LG Kulkas 2 pintu 5000000 NULL NULL Edit Copy Delete 3 AC Panasonic AC 1 PK 4000000 NULL NULL Edit Copy Delete 4 Laptop Asus Laptop gaming 12000000 NULL NULL Edit Copy Delete 5 HP Samsung Android terbaru 6000000 NULL NULL Edit Copy Delete 6 Mesin Cuci Front loading 4500000 NULL NULL Edit Copy Delete 7 Speaker JBL Bluetooth speaker 1500000 NULL NULL Edit Copy Delete 8 Monitor LG Monitor 24 inch 2500000 NULL NULL Edit Copy Delete 9 Keyboard Logitech Mechanical keyboard 800000 NULL NULL Edit Copy Delete 10 Mouse Razer Gaming mouse 600000 NULL NULL	-
3.	Buat model bernama Product php artisan make:model Product	PS C:\Users\MHS\laravel_api> php artisan make:model product INFO Model [C:\Users\MHS\laravel_api\app\Models\product.php] created successfully. PS C:\Users\MHS\laravel_api>	-
4.	Hubungkan model Product dengan tblproduct dengan mengisi pada class Product	app > Models > product.php 2 3 namespace App\Models; 4 5 use Illuminate\Database\Eloquent\Model; 6 7 class product extends Model 8 { 9 protected $table = 'tblproducts'; 10 11 protected $fillable = [12 'nama', 13 'deskripsi', 14 'foto', 15 'harga' 16]; 17 } 18	-

5.	Pada Product controller ubah dalam fungsi index, dapatkan data dari model Product Dengan menambahkan $products = Product:all(); Dan lakukan return data => $products	app > Http > Controllers > Api > ProductController.php
		1 <?php
		2
		3 namespace App\Http\Controllers\Api;
		4
		5 use App\Http\Controllers\Controller;
		6 use Illuminate\Http\Request;
		7
		8 class ProductController extends Controller
		9 {
		10 public function index()
		11 {
		12 $products = Product::all();
		13
		14 return response()->json([
		15 'data' => $products
		16]);
		17 }
		18 }
		19
			-

D.	**Membuat CRUD standar API Resource**		
1.	Ubah api.php dengan api Resource yang sesuai standar laravel	routes > api.php 1 <?php 2 3 use Illuminate\Support\Facades\Route; 4 use App\Http\Controllers\Api\ProductController; 5 6 // Route::get('/products', [ProductController::class, 'index']); 7 8 Route::apiResources('products', ProductController::class);	-

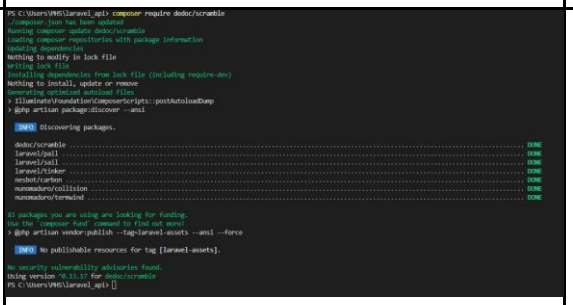
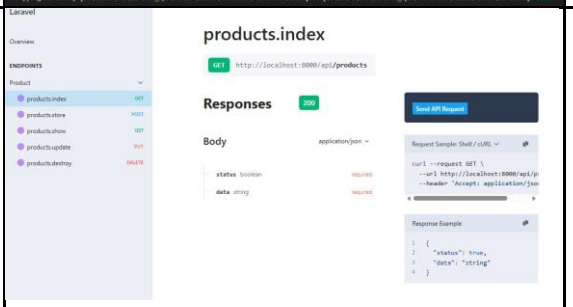
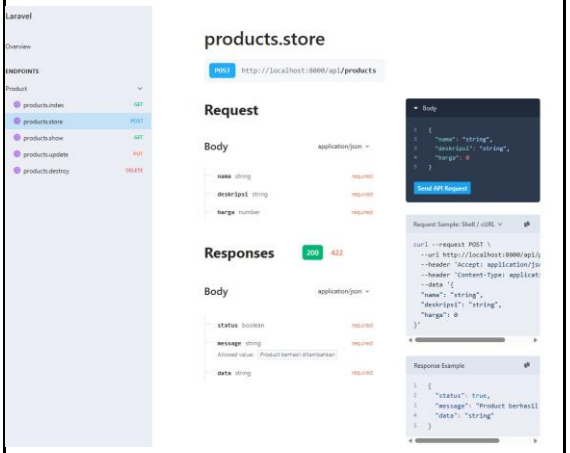
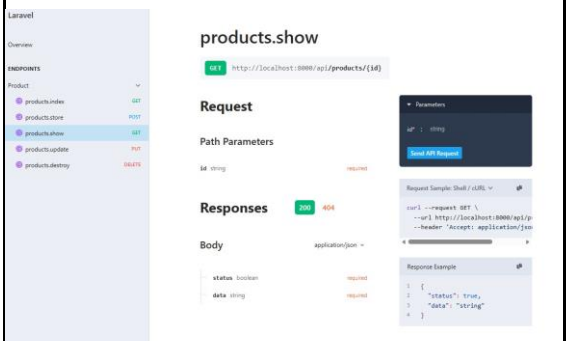
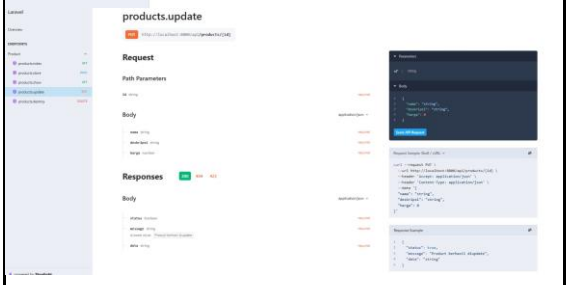
2.	2. Ubah ProductController untuk fungsi-fungsi berikut index, show, store, update, dan destroy.
	3. Fungsi Index pada ProductsController
		// GET /products
		public function index()
		{
		$products = Product::all();
		
		return response()->json([
		'status' => true,
		'data' => $products
		]);
		}
			-

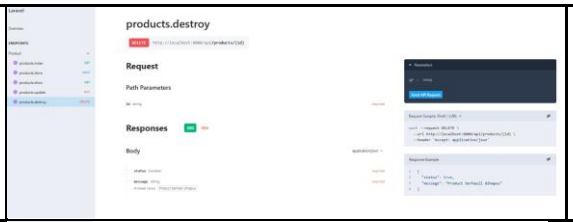
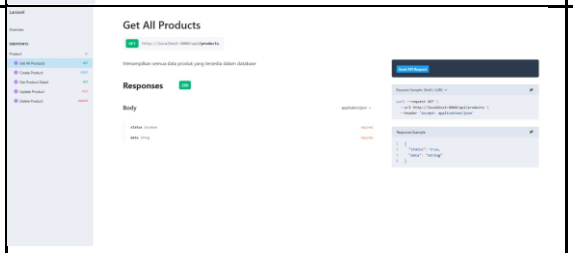
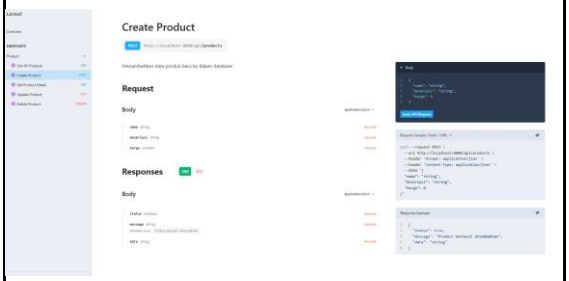
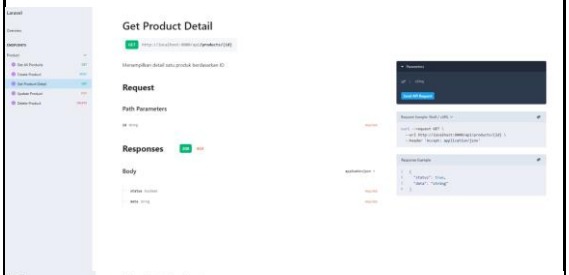
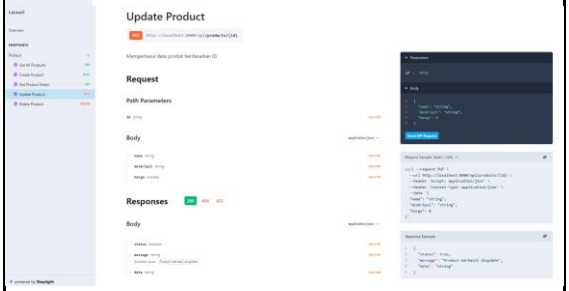
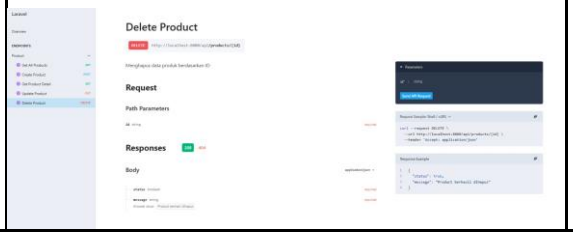
3.	Fungsi Show pada ProductsController	
		``
		// GET /products/{id}
		public function show($id)
		{
		$product = Product::find($id);
		
		if (!$product) {
		return response()->json([
		'status' => false,
		'message' => 'Product tidak ditemukan'
		], 404);
		}
		
		return response()->json([
		'status' => true,
		'data' => $product
		]);
		}
		``
		
			-

4.	Fungsi Store pada ProductsController	
		``
		//POST /products
		public function store(Request $request)
		{
		$request->validate([
		'nama' => 'required',
		'deskripsi' => 'required',
		'harga' => 'required|numeric',
		]);
		
		$product = Product::create($request->all());
		
		return response()->json([
		'status' => true,
		'message' => 'Product berhasil ditambahkan',
		'data' => $product
		]);
		}
		``
		
			-

5.	Fungsi Update pada ProductsController	
		``
		// PUT /products/{id}
		public function update(Request $request, $id)
		{
		$product = Product::find($id);
		if (!$product) {
		return response()->json([
		'status' => false,
		'message' => 'Product tidak ditemukan'
		], 404);
		}
		
		$request->validate([
		'nama' => 'required',
		'deskripsi' => 'required',
		'harga' => 'required|numeric',
		]);
		
		$product->update($request->all());
		return response()->json([
		'status' => true,
		'message' => 'Product berhasil diupdate',
		'data' => $product
		]);
		}
		``
		
			-

6.	Fungsi Destroy pada ProductsController	
		``
		// DELETE /products/{id}
		public function destroy($id)
		{
		$product = Product::find($id);
		
		if (!$product) {
		return response()->json([
		'status' => false,
		'message' => 'Product tidak ditemukan'
		], 404);
		}
		
		$product->delete();
		
		return response()->json([
		'status' => true,
		'message' => 'Product berhasil dihapus'
		]);
		}
		``
		
			-

E.	API Documentation		
1.	1. Install Scramble dengan menggunakan composer composer require dedoc/scramble 2. Masukan ke vendor laravel project anda php artisan vendor:publish --provider="Dedoc\Scramble\ScrambleServiceProvider"	 	-
2.	3. Jalankan server (artisan serve) 4. Akses API documentation pada browser ke http://localhost:8000/docs/api	    	-

			
3.	5. Untuk mendambahkan deksripsi setiap API Endpoints bisa menggunakan DocBlock diatas fungsi dalam controller 6. Menambahkan DocBlock (Title dan Description) 7. Refresh browser ke http://localhost:8000/docs/api 8. Tambahkan Title dan Deskripsi pada semua Endpoints	    	-